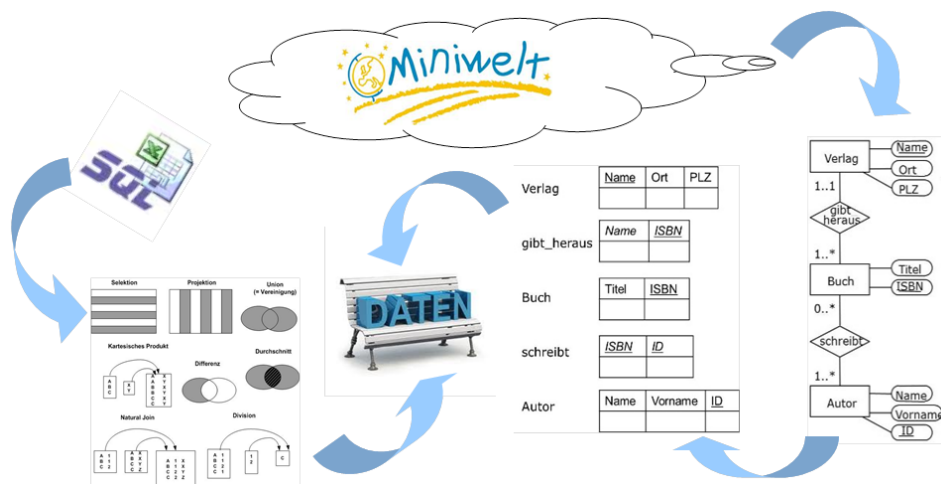


☒ DEM2 G1 Dr. Pitzer Name Klemens Danner Aufwand in h 5,5
☐ DEM2 G2 Dr. Pitzer
☐ DEM2 G3 Dr. Niklas Punkte _____ Kurzzeichen Tutor _____

Hinweise und Richtlinien:

- Übungsausarbeitungen müssen den im Syllabus angegebenen Formatierungsrichtlinien entsprechen – Nichtbeachtung dieser Formatierungsrichtlinien führt zu Punkteabzug.
- Zusätzlich zu den allgemeinen Formatierungsrichtlinien sind für diese Übungsausarbeitung folgende zusätzlichen Richtlinien zu beachten:
 - Treffen Sie, falls notwendig, sinnvolle Annahmen und dokumentieren Sie diese nachvollziehbar in ihrer Lösung!
 - Belegen Sie die Ausführung jedes Statements und führen Sie dazu die Ausgabe der Datenbank direkt beim jeweiligen Statement in Ihrer Übungsdokumentation an.

Ziel dieser Übung ist es, die Anfragesprache SQL im Bereich Datenmanipulation (Einfügen, Löschen, Ändern von Datensätzen) praktisch zu vertiefen sowie das Sicht-Konzept anzuwenden.



1. Daten bearbeiten – INSERT, UPDATE, DELETE (Human Resources) (10 Punkte)

Die Personalabteilung möchte, dass Sie SQL-Anweisungen für das Einfügen, Aktualisieren und Löschen von Angestelltendaten erstellen. Bevor Sie die Anweisungen an die Personalabteilung übergeben, erzeugen Sie die angegebenen Tabelle MY_EMPLOYEE als Hilfstabellen, um ihre SQL-Anweisungen zu testen.

```
CREATE TABLE MY_EMPLOYEE (  
  id NUMBER(6) CONSTRAINT my_employee_id_pk PRIMARY KEY,  
  last_name VARCHAR2(25) NOT NULL,  
  first_name VARCHAR2(20) NOT NULL,  
  email VARCHAR2(8),  
  salary NUMBER(8,2) NOT NULL,  
  hire_date DATE DEFAULT SYSDATE);
```

1. Erstellen Sie zwei INSERT-Anweisungen, um der Tabelle MY_EMPLOYEE die Zeilen 1 und 2 der folgenden Beispieldaten hinzuzufügen. Listen Sie die Spalten nicht in der INSERT-Klausel auf. (1 Punkt)

ID	LAST_NAME	FIRST_NAME	EMAIL	SALARY	HIRE_DATE
1	Fawcett	Matthew	mface	2895	gestern
2	Paltrow	Vivien		3860	01.05.2024
3	Peck	Fred	fpeck	1100	01.01.2025
4	Olivier	Kevin	kolivi	2750	15.02.2025
5	Dean	John	jdean	1550	

Füllen Sie die Tabelle MY_EMPLOYEE mit den restlichen Zeilen der Beispieldaten aus der Liste in Aufgabe 1 auf. Geben Sie dieses Mal die Spalten explizit in der INSERT-Klausel an.

Kontrollieren Sie die gerade eingefügten Daten und fragen Sie dazu die Tabelle ab. Schreiben Sie dann die Daten dauerhaft fest, verwenden Sie den Befehl COMMIT. (1,5 Punkte)

2. Die Angestellte mit der ID = 5 hat geheiratet, ändern Sie den Nachnamen auf „Voight“ und passen Sie die E-Mail Adresse an („jvoigh“). (1 Punkt)
3. Alle Angestellten deren Gehalt unter 2000 liegt, erhalten Gehaltserhöhungen. Die Erhöhung beträgt 10 % vom aktuellen Gehalt; weiters wurde das Mindestgehalt auf 1250 angepasst. Führen Sie diese Änderung mit einem Datenbank-Statement aus. (1 Punkt)
4. Legen Sie für alle Angestellten, die noch keine E-Mail Adresse haben, einen solchen fest. Die Adresse setzt sich aus dem ersten Buchstaben des Vornamens und maximal fünf Buchstaben des Nachnamens zusammen (Kleinbuchstaben). (1,5 Punkte)
5. Löschen Sie „Fred Peck“ aus der Tabelle MY_EMPLOYEE und schreiben Sie alle noch nicht gespeicherten Änderungen mit COMMIT fest. (0,5 Punkte)
6. Angestellte, die bereits mindestens zwei Job-Wechsel hinter sich haben, sollen einen Bonus von 500 bekommen. Allerdings dürfen Sie aktuell keinen Job (job_id) versehen, den Sie bereits in der Vergangenheit einmal übernommen hatten. Gehen Sie dabei folgendermaßen vor:
 - a. Befüllen Sie die Tabelle MY_EMPLOYEE mit den existierenden Datensätzen aus der Tabelle EMPLOYEES. Erstellen Sie eine geeignete INSERT-Anweisung zur Übernahme der Datensätze. Überprüfen Sie danach, ob alle Datensätze enthalten sind und schreiben Sie diese dauerhaft fest. (0,5 Punkte)
 - b. Erstellen Sie nun eine SQL-Anweisung, um jene Zeilen zu aktualisieren, die mindestens zwei Einträge in der Tabelle JOB_HISTORY aufweisen. Prüfen Sie auch, ob der aktuelle Job (EMPLOYEES.JOB_ID) nicht in der Tabelle JOB_HISTORY enthalten ist. Tipp: gehen Sie in zwei Schritten vor, erstellen Sie zuerst eine SELECT-Abfrage und im zweiten Schritt daraus das UPDATE. Überprüfen Sie anschließend die durchgeführte Aktualisierung. (1,5 Punkte)
7. Die Angestellten „Hunold“ und „De Haan“ werden pensioniert. Dabei entdecken Sie, dass sich auch noch Angestellte mit einem älteren Einstellungsdatum in der Datenbank befinden, die bereits alle in Pension sind. Bereinigen Sie alle Datensätze, deren Einstellungsdatum gleich oder vor diesen beiden Angestellten ist und löschen Sie diese mit einem DELETE-Befehl. Überprüfen Sie die Löschung mit einem SELECT. (1,5 Punkte)

2. Daten bearbeiten – MERGE (Datenbankschema: Human Resources)

(4 Punkte)

Erstellen Sie für die Aufgabe 2 die Tabelle BONUS mit folgendem Schema:

```
CREATE TABLE bonus (  
  employee_id NUMBER(6) CONSTRAINT bonus_employee_id_pk PRIMARY KEY,  
  bonus NUMBER(6,2) DEFAULT 100);
```

Fügen Sie in die Tabelle BONUS mit der angeführten INSERT-Anweisung Datensätze ein und schreiben Sie die eingefügten Datensätze mit COMMIT in der Datenbank dauerhaft fest:

```
INSERT INTO bonus (employee_id)  
(SELECT employee_id  
  FROM EMPLOYEES  
  WHERE department_id IN (50, 80));
```

```
COMMIT;
```

Die Personalabteilung möchte die Bonusbeträge aller Angestellten (Tabelle EMPLOYEES) für das abgelaufene Quartal neu berechnen und die existierende Tabelle BONUS mit den bereits enthaltenen Datensätzen heranziehen und entsprechend aktualisieren.

Das Management trifft folgende Vorgaben für die Bonusberechnung:

- Bonusbeträge werden generell nur jenen Angestellten zuerkannt, die ein Gehalt unter \$11.000 beziehen – dies trifft auch auf alle Angestellten zu, die bereits im vorangegangenen Quartal Bonusbeträge bezogen haben (siehe Tabelle BONUS) – wird hier die Höchstgrenze von \$11.000 überschritten (bspw. aufgrund der Gehaltserhöhung), so sind die Datensätze der jeweiligen Angestellten aus der Tabelle BONUS zu löschen.
- Wird ein Bonusbetrag erstmalig bzw. wieder zuerkannt (d.h., es existiert kein Eintrag in der Tabelle BONUS), so beträgt dieser 1 % des aktuellen Gehaltes (EMPLOYEES.salary).
- Angestellte, die bereits im vorangegangenen Quartal einen Bonusbetrag erhalten haben (d.h., es existiert ein Eintrag in der Tabelle BONUS) und die Gehaltsgrenze von \$11.000 nicht überschreiten bekommen eine Erhöhung um 15% des aktuellen Bonusbetrages.
- Angestellte der Abteilung mit der department_id = 80 werden die Bonusbeträge aberkannt, da sie die im abgelaufenen Quartal die vorgegebenen Quartalsziele nicht erreicht haben.
- Angestellte ohne Abteilung werden wie Angestellte mit Abteilung behandelt.

Erstellen Sie eine kompakte MERGE-Anweisung, die die geforderten Einzeloperationen (UPDATE, DELETE, INSERT) in einem Durchlauf ausführt. Prüfen Sie die Anweisung und die geänderten Daten.

3. Sichten (Datenbankschema: Human Resources)

(5 Punkte)

1. Für Schulungszwecke müssen Daten von Mitarbeiter*innen zur Verfügung gestellt werden; die vertraulichen Inhalte sind dabei auszublenden. Erstellen Sie eine Sicht, die nur folgende Daten enthält: Angestellten-Nummer, Vorname, Nachname, Abteilungsname und Bundesstaat (state_province). Wählen Sie auch passende Spaltenüberschriften und selektieren Sie den Inhalt Ihrer Sicht. (1 Punkt)
2. Die Abteilung 90 benötigt Zugriff auf ihre Angestellten Daten. Erstellen Sie eine Sicht mit dem Namen DEPT90, welche die Angestelltennummern, Nachnamen und Abteilungsnummern für alle Angestellten der Abteilung 90 enthält. Die Spalten der Sicht sollen mit EMPNO, EMPLOYEE und DEPTNO benannt werden. Sorgen Sie aus Sicherheitsgründen dafür, dass kein Angestellter über die Sicht einer anderen Abteilung neu zugeordnet werden kann (= Verhinderung der Tupelmigration). Selektieren Sie den Inhalt der Sicht DEPT90. (1 Punkt)

3. Testen Sie die Sicht DEPT90. (1 Punkt)

- a) Versuchen Sie, den Angestellten „Kochhar“ der Abteilung 60 zuzuordnen.
- b) Versuchen Sie, den Angestellten 100 auf „The King“ zu ändern.
- c) Kommentieren Sie das Ergebnis und nehmen Sie die Änderungen wieder zurück.

4. Erstellen Sie eine Sicht mit den Jahresgehältern für alle Angestellte. Verketteten Sie Vor- und Nachname mit einem Leerzeichen und multiplizieren Sie das Monatsgehalt mit 14; die Sicht soll auch die Angestellten-Nummer enthalten. Testen Sie verschiedene DML-Operationen auf der Sicht, verwenden Sie dazu z.B. die Angestellten 100 und 178 und kommentieren Sie das Ergebnis. Nehmen Sie anschließend Ihre Änderungen wieder zurück. (2 Punkte)

Hinweis zur Ausarbeitung: Sie können Änderungen nicht mehr zurücknehmen, wenn Sie ein DDL-Statement absetzen (dieses führt ein implizites COMMIT durch). Bei versehentlich festgeschriebenen Änderungen, laden Sie die HR-Datenbank erneut.

4. Korrelierte Anfragen

(3 Punkte)

1. Das folgende Statement gibt alle Informationen zu Angestellten aus, die vor ihrer*m (direkten) Manager*in eingestellt wurden. Entwickeln Sie eine nicht-korrelierte Variante. (1 Punkt)

```
SELECT *
FROM employees e
WHERE EXISTS (SELECT *
              FROM employees m
              WHERE e.manager_id = m.employee_id
              AND e.hire_date < m.hire_date);
```

2. Im folgenden Statement sind zwei korrelierte Unterabfragen enthalten. Wandeln Sie beide Unterabfragen so um, dass nur mehr nicht-korrelierte Unterabfragen enthalten sind. Beachten Sie, dass ein korreliertes Statement im SELECT ev. NULL-Werte zurückgeben könnte! (2 Punkte)

```
SELECT last_name, first_name, job_id, (SELECT department_name
                                       FROM departments
                                       WHERE department_id =
e1.department_id) AS department_name
FROM employees e1
WHERE salary > (SELECT AVG(salary)
               FROM employees
               WHERE job_id = e1.job_id);
```

5. Theoriefragen

(2 Punkte)

Kreuzen Sie alle der folgenden Aussagen an, die wahr sind. Bitte beachten Sie, dass Sie dazu möglicherweise auch außerhalb der zur Verfügung gestellten Unterlagen recherchieren müssen.

	richtig	falsch
Virtuelle Sichten werden für die Performance-Steigerung von Abfragen eingesetzt.	☐	☒
Mit der <code>WITH CHECK OPTION</code> werden vorhandene Check-Klausel der Basistabellen aktiviert.	☐	☒
Die Ergebnismenge einer virtuellen Sicht wird bei Ausführung erstellt und nicht dauerhaft persistiert.	☒	☐
Der Einsatz von <code>MERGE</code> ist vor allem dann sinnvoll, wenn sich Relationen nicht in der zweiten Normalform befinden.	☐	☒
Bei <code>TRUNCATE</code> dürfen keine aktiven <code>FOREIGN KEY</code> Constraints auf der Tabelle vorliegen.	☒	☐
Wurden Daten bereits mit <code>COMMIT</code> festgeschrieben, können sie weder mit einem <code>SAVEPOINT</code> noch mit einem <code>ROLLBACK</code> zurückgeholt werden.	☒	☐
Mehrere DML Statements werden vor der Ausführung vom SQL Optimizer in die richtige Reihenfolge gebracht.	☐	☒
Das Flashback Feature von Oracle erlaubt Abfragen auf einen bereits vergangenen Datenstand (Historie).	☒	☐

Aufgabe 1

1.1

Code

```
-- 1.1

INSERT INTO my_employee VALUES (1, 'Fawcett', 'Mattheuw', 'mface', 2895, SYSDATE - 1);
INSERT INTO my_employee VALUES (2, 'Paltrow', 'Vivien', NULL, 3860, TO_DATE('01.05.2024', 'DD.MM.YYYY'));

--1.2

INSERT INTO my_employee (id, last_name, first_name, email, salary, hire_date)
VALUES (3, 'Peck', 'Fred', 'fpeck', 1100, TO_DATE('01.01.2025', 'DD.MM.YYYY'));

INSERT INTO my_employee (id, last_name, first_name, email, salary, hire_date)
VALUES (4, 'Olivier', 'Kevin', 'kolivi', 2750, TO_DATE('15.02.2025', 'DD.MM.YYYY'));

INSERT INTO my_employee (id, last_name, first_name, email, salary, hire_date)
VALUES (5, 'Dean', 'John', 'jdean', 1550, NULL);

SELECT *
FROM my_employee;

COMMIT;
```

Ausgabe

		ID	LAST_NAME	FIRST_NAME	EMAIL	SALARY	HIRE_DATE
1 row inserted.	1	1	Fawcett	Mattheuw	mface	2895	09-JUN-26
1 row inserted.	2	2	Paltrow	Vivien	(null)	3860	01-MAY-24
1 row inserted.	3	3	Peck	Fred	fpeck	1100	01-JAN-25
1 row inserted.	4	4	Olivier	Kevin	kolivi	2750	15-FEB-25
1 row inserted.	5	5	Dean	John	jdean	1550	(null)
Commit complete.							

1.2

Code

```
-- 2 -- last_name von id 5 ändern, email ändern
```

```
UPDATE my_employee
SET last_name = 'Voight', email = 'jvoigh'
WHERE id = 5;
```

Ausgabe

1 row updated.

	ID	LAST_NAME	FIRST_NAME	EMAIL	SALARY	HIRE_DATE
1	1	Fawcett	Mattheuw	mface	2895	09-JUN-26
2	2	Paltrow	Vivien	(null)	3860	01-MAY-24
3	3	Peck	Fred	fpeck	1100	01-JAN-25
4	4	Olivier	Kevin	kolivi	2750	15-FEB-25
5	5	Voight	John	jvoigh	1550	(null)

1.3

Code

```
UPDATE my_employee
SET salary = GREATEST(salary * 1.1, 1250)
WHERE salary < 2000;
```

Ausgabe

2 rows updated.

	ID	LAST_NAME	FIRST_NAME	EMAIL	SALARY	HIRE_DATE
1	1	Fawcett	Mattheuw	mface	2895	09-JUN-26
2	2	Paltrow	Vivien	(null)	3860	01-MAY-24
3	3	Peck	Fred	fpeck	1250	01-JAN-25
4	4	Olivier	Kevin	kolivi	2750	15-FEB-25
5	5	Voight	John	jvoigh	1705	(null)

1.4

Code

```
UPDATE my_employee
SET email = LOWER(
  SUBSTR(first_name, 1, 1) || SUBSTR(last_name, 1, 5)
)
WHERE email IS NULL;
```

Ausgabe

1 row updated.

	ID	LAST_NAME	FIRST_NAME	EMAIL	SALARY	HIRE_DATE
1	1	Fawcett	Mattheuw	mface	2895	09-JUN-26
2	2	Paltrow	Vivien	vpaltr	3860	01-MAY-24
3	3	Peck	Fred	fpeck	1250	01-JAN-25
4	4	Olivier	Kevin	kolivi	2750	15-FEB-25
5	5	Voight	John	jvoigh	1705	(null)

1.5

Code

```
DELETE FROM my_employee
WHERE first_name = 'Fred' AND last_name = 'Peck';

COMMIT;
```

Ausgabe

	ID	LAST_NAME	FIRST_NAME	EMAIL	SALARY	HIRE_DATE
1	1	Fawcett	Mattheuw	mface	2895	09-JUN-26
2	2	Paltrow	Vivien	vpaltr	3860	01-MAY-24
3	4	Olivier	Kevin	kolivi	2750	15-FEB-25
4	5	Voight	John	jvoigh	1705	(null)

1 row deleted.

1.6a

Code

```
INSERT INTO my_employee (id, last_name, first_name, email, salary, hire_date)
SELECT employee_id, last_name, first_name, email, salary, hire_date
FROM employees;

SELECT *
FROM my_employee;

COMMIT;
```

Ausgabe

ID	LAST_NAME	FIRST_NAME	EMAIL	SALARY	HIRE_DATE	
1	1	Fawcett	Mattheuw	mface	2895	09-JUN-26
2	2	Paltrow	Vivien	vpaltr	3860	01-MAY-24
3	4	Olivier	Kevin	kolivi	2750	15-FEB-25
4	5	Voight	John	jvoigh	1705	(null)
5	100	King	Steven	SKING	24000	17-JUN-87
6	101	Kochhar	Neena	NKOCHHAR	17000	21-SEP-89
7	102	De Haan	Lex	LDEHAAN	17000	13-JAN-93
8	103	Hunold	Alexander	AHUNOLD	9000	03-JAN-98
9	104	Ernst	Bruce	BERNST	6000	21-MAY-91
10	107	Lorentz	Diana	DLORENTZ	4200	07-FEB-99
11	124	Mourgos	Kevin	KMOURGOS	5800	16-NOV-99
12	141	Rajs	Trenna	TTRAJS	3500	17-OCT-95
13	142	Davies	Curtis	CDAVIES	3100	29-JAN-97
14	143	Matas	Randall	RMATOS	2600	15-MAR-98
15	144	Vargas	Peter	PVARGAS	2500	09-JUL-98
16	149	Zlotkey	Eleni	EZLOTKEY	10500	29-JAN-00
17	174	Abel	Ellen	EABEL	11000	11-MAY-96
18	176	Taylor	Jonathon	JTAYLOR	8600	24-MAR-98
19	178	Grant	Kimberely	KGRANT	7000	24-MAY-99
20	200	Whalen	Jennifer	JWHALEN	4400	17-SEP-87
21	201	Hartstein	Michael	MHARTSTE	13000	17-FEB-96
22	202	Fay	Pat	PFAY	6000	17-AUG-97
23	205	Higgins	Shelley	SHIGGINS	12000	07-JUN-94
24	206	Gietz	William	WGIETZ	8300	07-JUN-94

20 rows inserted.

Commit complete.

1.6b

Code

```

UPDATE my_employee
SET salary = salary + 500
WHERE id IN (
  SELECT e.id
  FROM my_employee e JOIN employees e_orig ON (e.id = e_orig.employee_id)
  WHERE e.id IN (
    SELECT employee_id
    FROM job_history
    GROUP BY employee_id
    HAVING COUNT(*) >= 2
  )
  AND e_orig.job_id NOT IN ( -- Alle jobs die e_orig.employee_id = e.id in der vergangenheit gemacht hat
    SELECT job_id
    FROM job_history jh
    WHERE e.id = jh.employee_id
    GROUP BY job_id
  )
);

```

Ausgabe

1 row updated.

	ID	LAST_NAME	FIRST_NAME	EMAIL	SALARY	HIRE_DATE
1	1	Fawcett	Mattheuw	mface	2095	09-JUN-26
2	2	Paltrow	Vivien	vpaltr	3860	01-MAY-24
3	4	Olivier	Kevin	kolivi	2750	15-FEB-25
4	5	Voight	John	jvoigh	1705	(null)
5	100	King	Steven	SKING	24000	17-JUN-87
6	101	Kochhar	Neena	NKOCHHAR	17500	21-SEP-89
7	102	De Haan	Lex	LDEHAAN	17000	13-JAN-93
8	103	Hunold	Alexander	AHUNOLD	9000	03-JAN-90
9	104	Ernst	Bruce	BERNST	6000	21-MAY-91
10	107	Lorentz	Diana	DLORENTZ	4200	07-FEB-99
11	124	Mourgos	Kevin	KMOURGOS	5800	16-NOV-99
12	141	Rajs	Trenna	TRAJS	3500	17-OCT-95
13	142	Davies	Curtis	CDAVIES	3100	29-JAN-97
14	143	Matos	Randall	RMATOS	2600	15-MAR-98
15	144	Vargas	Peter	PVARGAS	2500	09-JUL-98
16	149	Zlotkey	Eleni	EZLOTKEY	10500	29-JAN-00
17	174	Abel	Ellen	EABEL	11000	11-MAY-96
18	176	Taylor	Jonathon	JTAYLOR	8600	24-MAR-98
19	178	Grant	Kimberely	KGRANT	7000	24-MAY-99
20	200	Whalen	Jennifer	JWHALEN	4400	17-SEP-87
21	201	Hartstein	Michael	MHARTSTE	13000	17-FEB-96
22	202	Fay	Pat	PFAY	6000	17-AUG-97
23	205	Higgins	Shelley	SHIGGINS	12000	07-JUN-94
24	206	Gietz	William	WGIEZT	8300	07-JUN-94

1.7

Code

```
DELETE FROM my_employee
WHERE hire_date <= (
  SELECT MAX(hire_date)
  FROM my_employee
  WHERE last_name = 'Hunold' OR last_name = 'De Haan'
);
```

Ausgabe

6 rows deleted.

	ID	LAST_NAME	FIRST_NAME	EMAIL	SALARY	HIRE_DATE
1	1	Fawcett	Mattheuw	mface	2895	09-JUN-26
2	2	Paltrow	Vivien	vpaltr	3860	01-MAY-24
3	4	Olivier	Kevin	kolivi	2750	15-FEB-25
4	5	Voight	John	jvoigh	1705	(null)
5	107	Lorentz	Diana	DLORENTZ	4200	07-FEB-99
6	124	Mourgos	Kevin	KMOURGOS	5800	16-NOV-99
7	141	Rajs	Trenna	TRAJS	3500	17-OCT-95
8	142	Davies	Curtis	CDAVIES	3100	29-JAN-97
9	143	Matos	Randall	RMATOS	2600	15-MAR-98
10	144	Vargas	Peter	PVARGAS	2500	09-JUL-98
11	149	Zlotkey	Eleni	EZLOTKEY	10500	29-JAN-00
12	174	Abel	Ellen	EABEL	11000	11-MAY-96
13	176	Taylor	Jonathon	JTAYLOR	8600	24-MAR-98
14	178	Grant	Kimberely	KGRANT	7000	24-MAY-99
15	201	Hartstein	Michael	MHARTSTE	13000	17-FEB-96
16	202	Fay	Pat	PFAY	6000	17-AUG-97
17	205	Higgins	Shelley	SHIGGINS	12000	07-JUN-94
18	206	Gietz	William	WGIETZ	8300	07-JUN-94

Aufgabe 2

2.1

Code

```

MERGE INTO bonus b
USING employees e
  ON (e.employee_id = b.employee_id)
WHEN MATCHED THEN -- Mitarbeiter existiert in Bonus
                  -- department_id = 80 werden gelöscht
                  -- über 11000€ Leute werden gelöscht
                  -- Erhöhung um 15% des aktuellen Bonusbetrages
  UPDATE SET b.bonus = b.bonus * 1.15
  DELETE WHERE (e.salary > 11000) OR (department_id = 80)

WHEN NOT MATCHED THEN -- Mitarbeiter hat noch keinen bonus erhalten
                      -- Bonus beträgt 1 % des Gehalts (e.salary)
                      -- department_id = 80 werden nicht eingefügt
                      -- Gehalt über 11000 wird nicht eingefügt
                      -- Null soll einen Bonus bekommen können
  INSERT (employee_id, bonus)
  VALUES (e.employee_id, e.salary * 0.01)
  WHERE ((e.department_id != 80) OR (e.department_id IS NULL)) AND (e.salary <= 11000);

```

Ausgabe

15 rows merged.

	EMPLOYEE_ID	BONUS
1	124	115
2	141	115
3	142	115
4	143	115
5	144	115
6	103	90
7	104	60
8	107	42
9	178	70
10	200	44
11	202	60
12	206	83

Aufgabe 3

3.1

Code

```
-- 1
CREATE OR REPLACE VIEW schulung (Personalnummer, Vorname, Nachname, Abteilung, Bundesland) AS
SELECT employee_id, first_name, last_name, department_name, state_province
FROM employees
  JOIN departments USING (department_id)
  JOIN locations USING (location_id);

SELECT * FROM schulung;
```

Ausgabe

View SCHULUNG created.

	PERSONALNUMMER	VORNAME	NACHNAME	ABTEILUNG	BUNDESLAND
1	100	Steven	King	Executive	Washington
2	101	Neena	Kochhar	Executive	Washington
3	102	Lex	De Haan	Executive	Washington
4	103	Alexander	Hunold	IT	Texas
5	104	Bruce	Ernst	IT	Texas
6	107	Diana	Lorentz	IT	Texas
7	124	Kevin	Mourgos	Shipping	California
8	141	Trenna	Rajs	Shipping	California
9	142	Curtis	Davies	Shipping	California
10	143	Randall	Matos	Shipping	California
11	144	Peter	Vargas	Shipping	California
12	149	Eleni	Zlotkey	Sales	Oxford
13	174	Ellen	Abel	Sales	Oxford
14	176	Jonathon	Taylor	Sales	Oxford
15	200	Jennifer	Whalen	Administration	Washington
16	201	Michael	Hartstein	Marketing	Ontario
17	202	Pat	Fay	Marketing	Ontario
18	205	Shelley	Higgins	Accounting	Washington
19	206	William	Gietz	Accounting	Washington

3.2

Code

```
CREATE OR REPLACE VIEW DEPT90 (EMPNO, EMPLOYEE, DEPTNO) AS
SELECT employee_id, last_name, department_id
FROM employees
WHERE department_id = 90
WITH CHECK OPTION CONSTRAINT dept90_department_id_check;

SELECT * FROM DEPT90;
```

Ausgabe

View DEPT90 created.

	EMPNO	EMPLOYEE	DEPTNO
1	100	King	90
2	101	Kochhar	90
3	102	De Haan	90

3.3a

Code

```
UPDATE dept90
SET deptNo = 60
WHERE employee = 'Kochhar';
```

Ausgabe

```
Error starting at line : 1 in command -
UPDATE dept90
SET deptNo = 60
WHERE employee = 'Kochhar'
Error at Command Line : 1 Column : 8
Error report -
SQL Error: ORA-01402: view WITH CHECK OPTION where-clause violation
```

3.3b

Code

```
UPDATE dept90
SET employee = 'The King'
WHERE empNo = 100;
```

Ausgabe

```
1 row updated.
```

	EMPNO	EMPLOYEE	DEPTNO
1	100	The King	90
2	101	Kochhar	90
3	102	De Haan	90

3.3c

Warum funktioniert a) nicht und b) schon?

- Durch die Änderung der Abteilungsnummer auf 60 würde der Datensatz aus der View rausfallen. Genau das verhindert die check-option.
 - Durch die Änderung von 'King' auf 'The King' fällt dieser Datensatz nicht aus der Sicht - daher wird das statement ausgeführt.
- Durch ein 'ROLLBACK' wurde die Änderung rückgängig gemacht.

	EMPNO	EMPLOYEE	DEPTNO
1	100	King	90
2	101	Kochhar	90
3	102	De Haan	90

3.4

Code

```
CREATE OR REPLACE VIEW annual_salary (employee, employee_id, annual_salary) AS
SELECT first_name || ' ' || last_name, employee_id, salary * 14
FROM employees;
```

```
SELECT * FROM annual_salary;
```

Ausgabe

View ANNUAL_SALARY created.

	EMPLOYEE	EMPLOYEE_ID	ANNUAL_SALARY
1	Steven King	100	336000
2	Neena Kochhar	101	238000
3	Lex De Haan	102	238000
4	Alexander Hunold	103	126000
5	Bruce Ernst	104	84000
6	Diana Lorentz	107	58800
7	Kevin Mourgös	124	81200
8	Trenna Rajs	141	49000
9	Curtis Davies	142	43400
10	Randall Matos	143	36400
11	Peter Vargas	144	35000
12	Eleni Zlotkey	149	147000
13	Ellen Abel	174	154000
14	Jonathon Taylor	176	120400
15	Kimberely Grant	178	98000
16	Jennifer Whalen	200	61600
17	Michael Hartstein	201	182000
18	Pat Fay	202	84000
19	Shelley Higgins	205	168000
20	William Gietz	206	116200

3.4 DMLs testen bei Angestellten 100 und 178

Code

```
UPDATE annual_salary
SET employee = 'Stephan König'
WHERE employee_id = 100;
```

```
UPDATE annual_salary
SET annual_salary = 0
WHERE employee_id = 100;
```

```
UPDATE annual_salary
SET employee_id = 50
WHERE employee_id = 178;
```

Ausgabe

Error starting at line : 1 in command - UPDATE annual_salary SET annual_salary = 0 WHERE employee_id = 100 Error at Command Line : 2 Column : 5 Error report - SQL Error: ORA-01733: virtual column not allowed here	Error starting at line : 1 in command - UPDATE annual_salary SET employee = 'Stephan König' WHERE employee_id = 100 Error at Command Line : 2 Column : 5 Error report - SQL Error: ORA-01733: virtual column not allowed here
14 Jonathon Taylor 176 120400	
15 Kimberely Grant 50 98000	
16 Jennifer Whalen 200 61600	

Note: Das erste statement kann nicht funktionieren, weil 'employee' aus mehreren Attributen zusammengebaut ist. Es wäre nicht klar, was genau in der Basistabelle geändert werden muss.

Auch das annual_salary anzupassen funktioniert nicht, weil im Select auf salary*14 projiziert wird.

Die employee_id wird direkt aus der Haupttabelle "weitergegeben" und ist daher änderbar, wenn kein Fremdschlüssel auf ihn referenziert.

Note: employee 178 ist jetzt employee 50

Code
<pre>-- delete DELETE FROM annual_salary WHERE employee_id = 100; DELETE FROM annual_salary WHERE employee_id = 50;</pre>
Ausgabe
<pre>Error starting at line : 1 in command - DELETE FROM annual_salary WHERE employee_id = 100 Error report - ORA-02292: integrity constraint (S2510307051.DEPT_MGR_FK) violated - child record found https://docs.oracle.com/error-help/db/ora-02292/ More Details : https://docs.oracle.com/error-help/db/ora-02292/ 1 row deleted.</pre>

Note: Datensatz mit der id 100 zu löschen funktioniert nicht, weil auf diesen als Fremdschlüssel referenziert wird (z.B. steht er bei jemand anderen als manager)

Bei Datensatz mit der id 50 (vorher 178) ist das nicht der Fall, deshalb lässt er sich löschen.

Aufgabe 4

4.1

Code

```
SELECT e.*
FROM employees e
JOIN employees m ON (e.manager_id = m.employee_id)
WHERE e.hire_date < m.hire_date;
```

Ausgabe

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	200	Jennifer	Whalen	JWHALEN	515.123.4444	17-SEP-87	AD_ASST	4400	(null)	101	10
2	103	Alexander	Hunold	AHUNOLD	590.423.4567	03-JAN-90	IT_PROG	9000	(null)	102	60
3	142	Curtis	Davies	CDAVIES	650.121.2994	29-JAN-97	ST_CLERK	3100	(null)	124	50
4	143	Randall	Matos	RMATOS	650.121.2874	15-MAR-98	ST_CLERK	2600	(null)	124	50
5	144	Peter	Vargas	PVARGAS	650.121.2004	09-JUL-98	ST_CLERK	2500	(null)	124	50
6	141	Trenna	Rajs	TRAJS	650.121.8009	17-OCT-95	ST_CLERK	3500	(null)	124	50
7	178	Kimberely	Grant	KGRANT	011.44.1644.429263	24-MAY-99	SA_REP	7000	0.15	149	(null)
8	174	Ellen	Abel	EABEL	011.44.1644.429267	11-MAY-96	SA_REP	11000	0.3	149	80
9	176	Jonathon	Taylor	JTAYLOR	011.44.1644.429265	24-MAR-98	SA_REP	8600	0.2	149	80

4.2

Code

```
SELECT e.last_name, e.first_name, e.job_id, d.department_name
FROM employees e
LEFT JOIN departments d ON (e.department_id = d.department_id)
-- employee bleibt erhalten, wenn department_id Null ist
JOIN (
SELECT job_id, AVG(salary) AS average_salary
FROM employees
GROUP BY job_id
) avg_sal ON (e.job_id = avg_sal.job_id)
WHERE e.salary > avg_sal.average_salary;
```

Ausgabe

	LAST_NAME	FIRST_NAME	JOB_ID	DEPARTMENT_NAME
1	Rajs	Trenna	ST_CLERK	Shipping
2	Davies	Curtis	ST_CLERK	Shipping
3	Hunold	Alexander	IT_PROG	IT
4	Abel	Ellen	SA_REP	Sales